

AI WEATHERWISE

RESTful Backend for Intelligent Weather Forecasts and AI-Powered Insights

Project Type	Node.js / Express.js REST API
Database	MongoDB with Mongoose ODM
AI	Google Gemini
Weather API	OpenWeatherMap
Authentication	JWT + bcrypt

Prepared for Skill Wallet submission

Contents

No.	Section
1	Technical Architecture
2	ER Diagram
3	Features
4	Roles and Responsibilities
5	User Flow
6	MVC Pattern
7	Creating the Project Folder
8	Server Setup and Configuration
9	Backend Structure
10	Development and Explanation
11	Configure MongoDB
12	Create Database Connection
13	Create Schema and Models
14	Steps for Project Execution
15	Output / API Testing Evidence
16	Demo Video
17	Reference Notes and Security

1. Technical Architecture

AI WeatherWise is a RESTful backend platform built with Node.js and Express.js. MongoDB with Mongoose provides data persistence, while live weather metrics are integrated with Google Gemini to produce intelligent summaries and personalized recommendations. The reference also describes JWT authentication, bcrypt password hashing, centralized error handling, request sanitization, and a fallback mode.

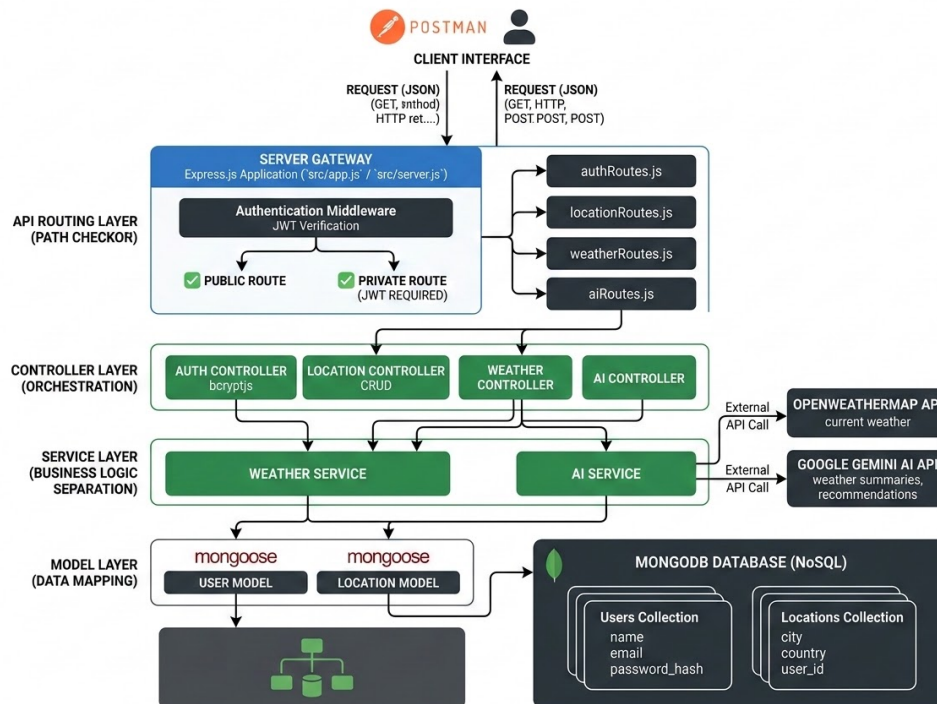


Figure 1. Architecture diagram supplied in the reference document.

- API interface and JSON responses.
- Express server gateway for HTTP methods, JSON parsing, CORS and routing.
- Authentication/authorization middleware for protected routes.
- Mongoose database interface for MongoDB.
- Weather integration through OpenWeatherMap.
- Gemini integration for summaries and recommendations.

2. ER Diagram

The reference defines User and Location as the main entities. One registered User can save multiple favorite Locations, forming a one-to-many relationship.

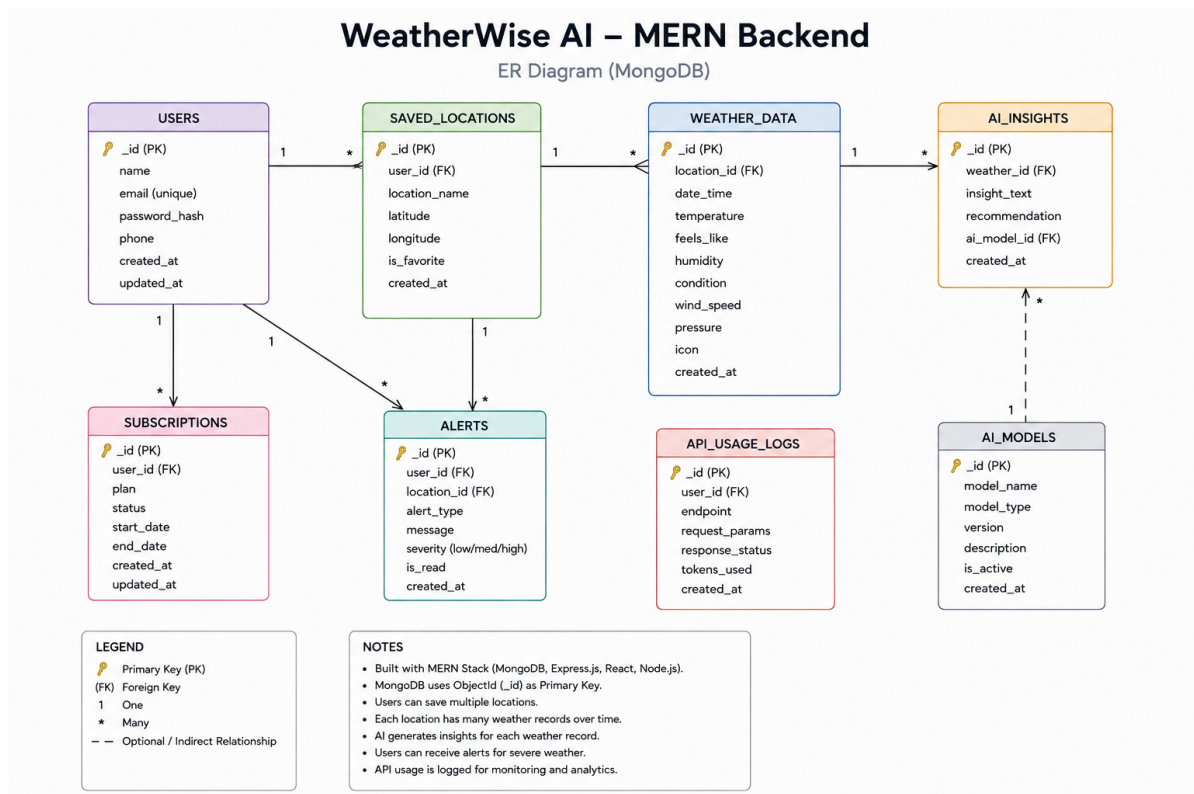


Figure 2. ER diagram supplied in the reference document.

User fields: _id, name, email, password, role.

Location fields: _id, user, city, country, createdAt.

3. Features

- Real-time weather retrieval.
- Gemini-generated natural-language weather summaries.
- Personalized activity and clothing recommendations.
- Favorite-location CRUD management.
- JWT authentication and bcrypt password hashing.
- Role-based route guarding.
- Request sanitization against the risks described in the reference.
- Centralized error handling.
- Fallback behavior when external API keys are unavailable.
- Public weather endpoints as described in the reference.

Case-study problem and solution

The reference presents a traveler who needs to track multiple cities, understand weather without reading raw metrics, and receive activity suggestions. AI WeatherWise centralizes weather retrieval, user/location storage, and Gemini-based insight generation.

4. Roles and Responsibilities

Role	Responsibilities
Admin	User account management; API-health/system-log oversight; database-integrity monitoring.
Registered User	Manage favorite locations; request weather; request Gemini summaries/recommendations; manage authentication details.
Public / Weather Application User	Consume public current-weather endpoints without an account, as described in the reference.

5. User Flow

AI WeatherWise API User Workflow

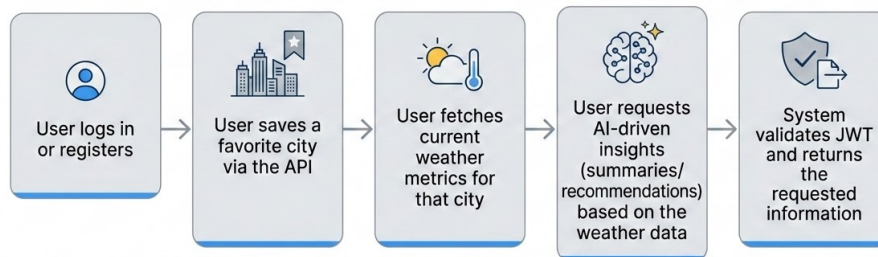


Figure 3. User workflow supplied in the reference.

1. User registers or logs in.
2. User saves a favorite city through the API.
3. User fetches current weather metrics.
4. User requests AI-driven summary/recommendation.
5. System validates JWT and returns the requested information.

6. MVC Pattern

The reference describes an MVC-style separation. Because this is a headless backend, the View is represented by the API routing/interface layer rather than a conventional HTML template.

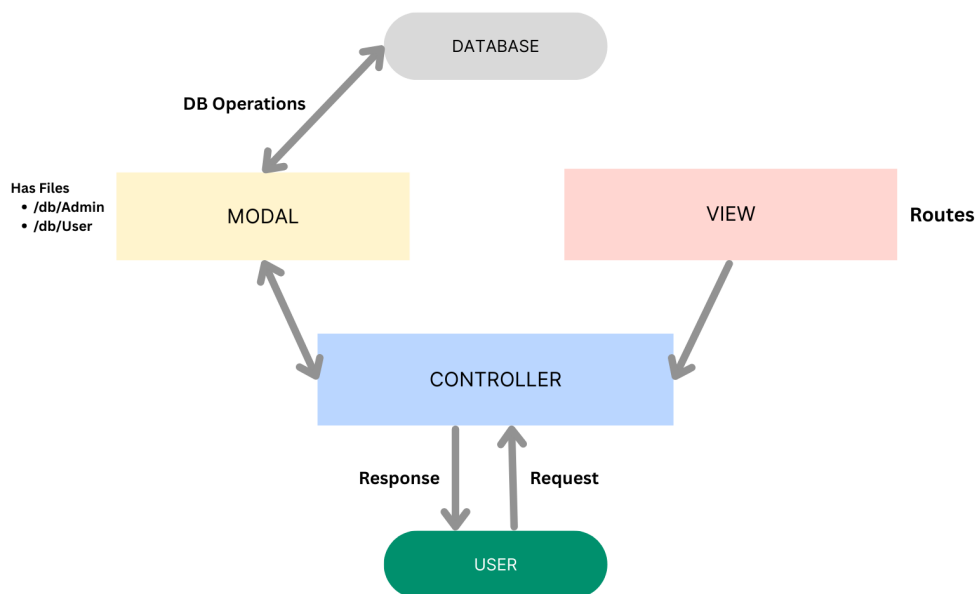


Figure 4. MVC diagram supplied in the reference.

Model

Mongoose schemas define the structure and persistence mapping for MongoDB documents.

Controller

Controllers receive requests, validate structure, delegate processing, and return JSON responses.

View / API Interface

Routes connect HTTP requests to controller handlers; API clients such as Postman or Thunder Client act as the practical interface.

7. Creating the Project Folder

The reference instructs the developer to create a folder named **AI Weatherwise**, open it in Visual Studio Code, initialize the Node.js project, and install dependencies.

```
npm init -y
```

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
```

```
npm install --save-dev nodemon
```

Root files listed by the reference: index.js, .env, package.json, .gitignore.

8. Server Setup and Configuration

The reference keeps configuration in environment variables.

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
GEMINI_API_KEY=your_gemini_api_key
```

The reference text lists port 5000, while the supplied API-testing screenshots use localhost:5001. The port should therefore be treated as configurable for the actual project.

```
const app = require('./app');
const connectDB = require('./config/db');

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      console.log('Press Ctrl+C to terminate.');
```

Figure 5. Server-start logic from the reference.

9. Backend Structure

The backend is described as modular, separating routing, controllers, middleware, models and utility modules for maintainability and scalability.

Layer	Purpose
routes	API route paths and controller connections.
controllers	Request handling and response logic.
middleware	Authentication, authorization, validation and sanitization.
models	Mongoose schemas and MongoDB data models.
utils / services	Reusable logic and external integrations.
index.js	Loads configuration, connects to MongoDB, registers routes and starts the server.

A supplied screenshot contains the unrelated visible project name “AI-STUDY-BUDDY”; it is intentionally omitted from this final document rather than presenting it as WeatherWise.

10. Development and Explanation

1. Load environment configuration.
2. Initialize and configure Express.
3. Connect to MongoDB using Mongoose.
4. Register middleware and API routes.
5. Authenticate users and issue/validate JWTs.
6. Retrieve weather information.
7. Send relevant weather conditions to Gemini for AI insights.
8. Return structured JSON responses.

MongoDB connection implementation evidence

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Figure 6. Database connection module shown in the reference.

11. Configure MongoDB

MongoDB is the NoSQL database used for persistence, with Mongoose acting as the ODM.

1. Install Mongoose.
2. Prepare the MongoDB connection string.
3. Place it in MONGO_URI.
4. Create the connection module using `mongoose.connect()`.
5. Ensure the database is connected before backend actions are processed.

```
npm install mongoose
```

The supplied code uses `process.env.MONGO_URI` with a local MongoDB fallback, logs the connected host, and exits if connection setup fails.

12. Create Database Connection

The connection module establishes communication between the application and MongoDB, reports the connection status, and handles connection errors.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Figure 7. Database connection implementation.

Responsibilities

- Read the MongoDB URI from configuration.
- Call `mongoose.connect()`.
- Log the connected MongoDB host.
- Stop the application when database connection fails.

13. Create Schema and Models

User model: `_id`, name, email, password and role. The reference marks name and email as required, email as unique, password as hashed/required, and role with a default value of reader.

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\.w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.w{2,3})+$/,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
    required: [true, 'Please provide a password'],
    minlength: [6, 'Password must be at least 6 characters']
  }
}, {
  timestamps: true
});
```

Figure 8. User schema evidence.

Location model: `_id`, user reference, city, country and createdAt.

```
const mongoose = require('mongoose');

const locationSchema = new mongoose.Schema({
  city: {
    type: String,
    required: [true, 'Please provide a city name'],
    trim: true
  },
  country: {
    type: String,
    required: [true, 'Please provide a country name'],
    trim: true
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'User reference is required']
  }
}, {
  timestamps: true
});

// Ensure a user cannot duplicate the exact same city and country in their favorites
locationSchema.index({ user: 1, city: 1, country: 1 }, { unique: true });

const Location = mongoose.model('Location', locationSchema);
module.exports = Location;
```

Figure 9. Location schema evidence.

14. Steps for Project Execution

1. Open the AI WeatherWise project in Visual Studio Code.
2. Open the integrated terminal.
3. Run npm install.
4. Verify MongoDB and environment variables.
5. Start the development server with npm start, as specified in the reference.
6. Wait for successful database connection and server startup.
7. Use Postman or Thunder Client for API testing.
8. Test authentication, weather retrieval and AI recommendation endpoints.

The supplied API evidence shows the local project being tested on localhost:5001.

15. Output Screenshots

API testing output from the completed AI WeatherWise backend.

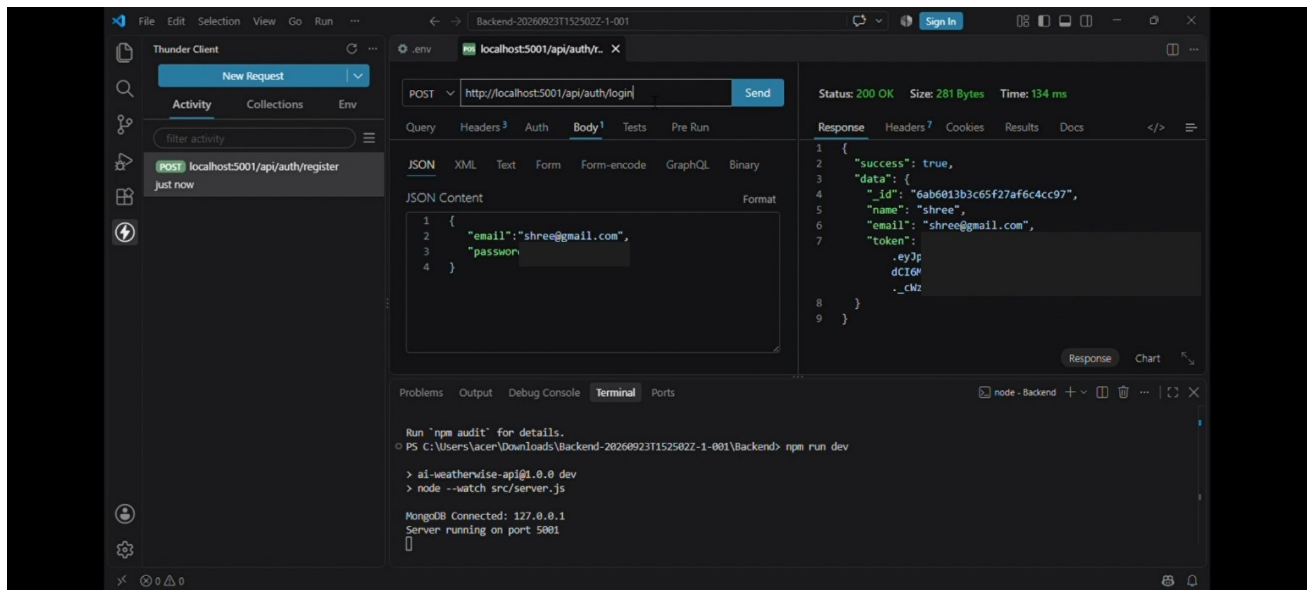


Figure 10. Successful user login API response.

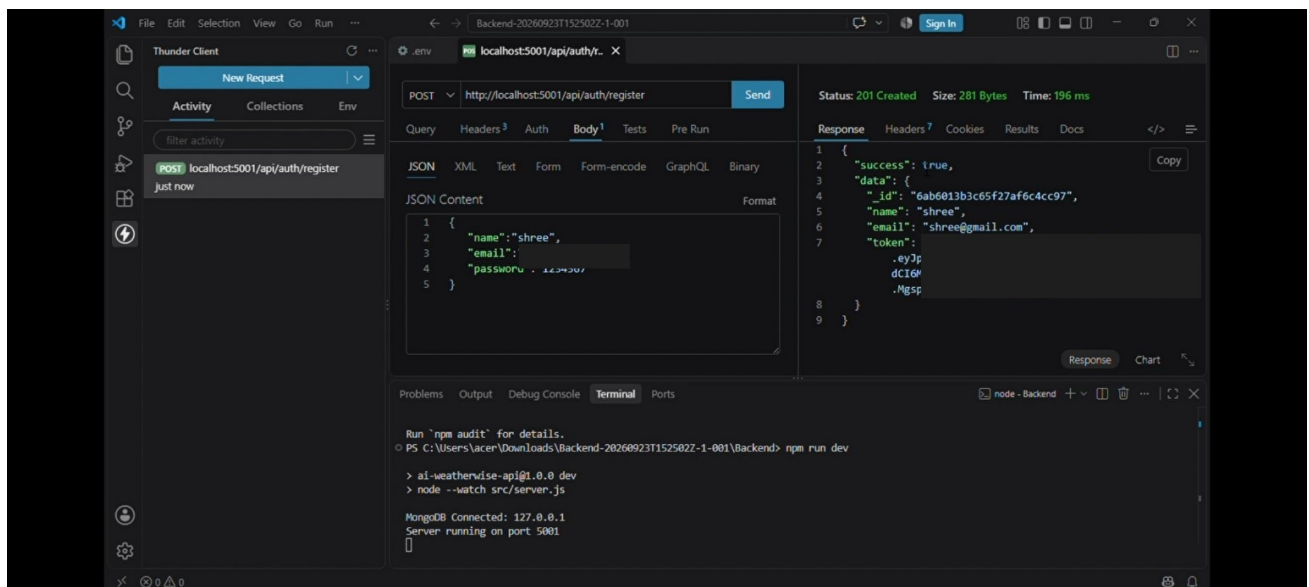


Figure 11. Successful user registration API response.

15. Output Screenshots (continued)

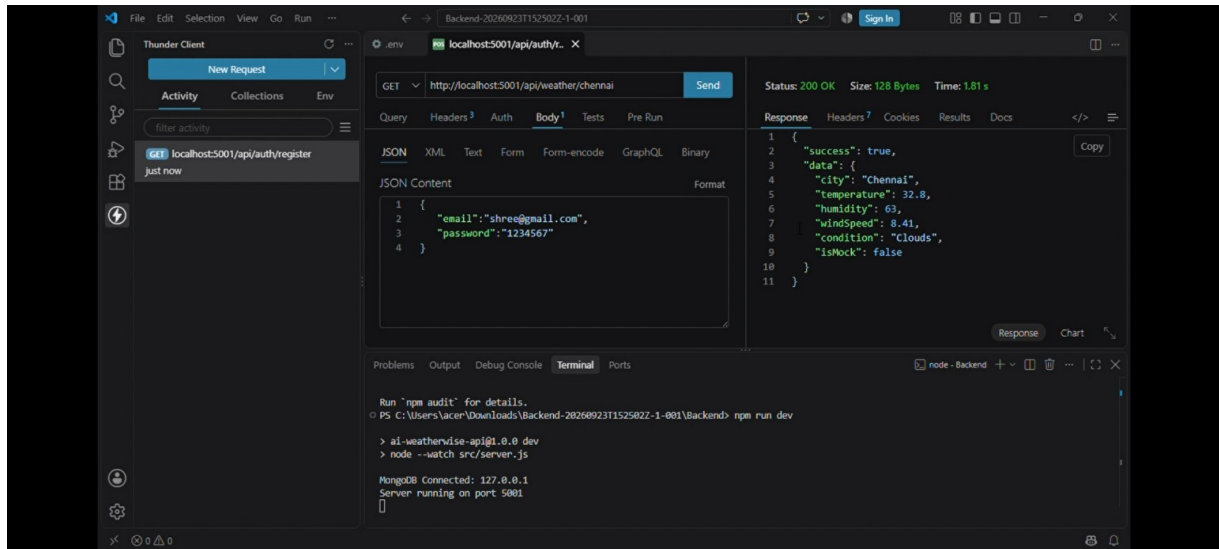


Figure 12. Successful current-weather API response for Chennai.

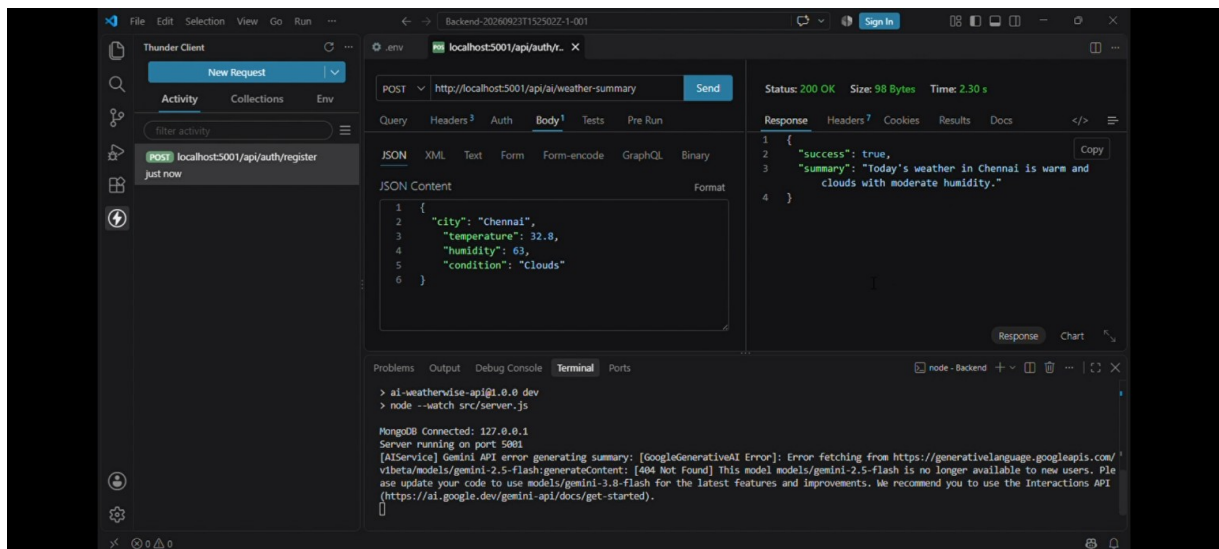


Figure 13. AI-generated weather summary response.

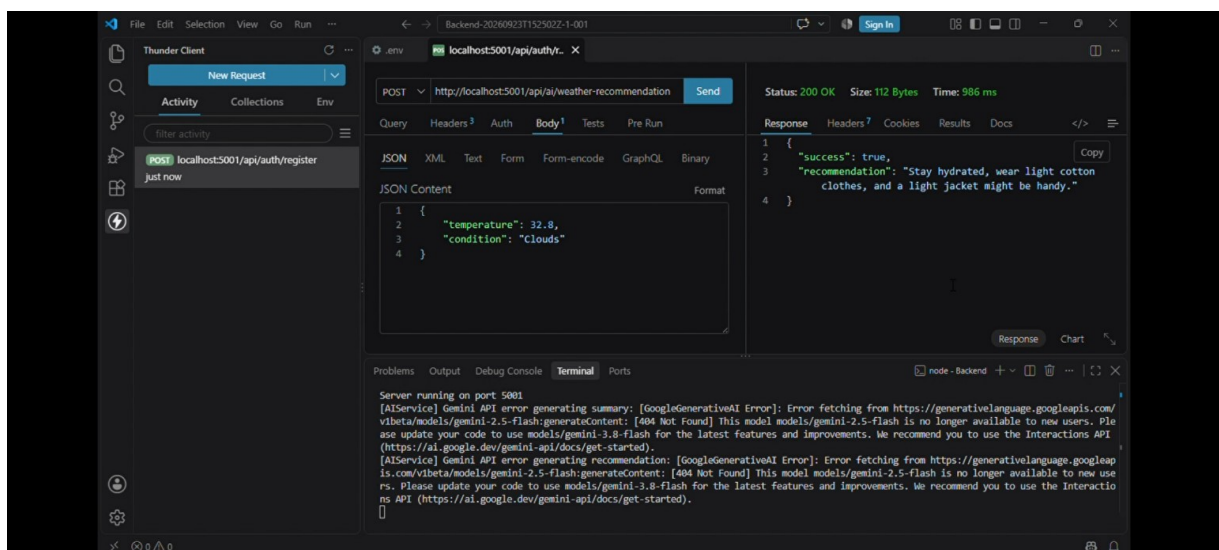


Figure 14. AI-generated weather recommendation response.

16. Demo Video

Demo Video Link

<https://drive.google.com/file/d/19eVCBGGt1dLXyAayzHNrWv58UmMIMcMu/view?usp=sharing>

A. Reference Notes and Security

This documentation is based on the supplied 15-page WeatherWise AI reference document and follows its terminology and sequence where the source provides sufficient information.

Source-consistency notes

- The reference lists PORT=5000 in the environment example, while the supplied API-testing screenshots use localhost:5001.
- Some reference paragraphs contain unrelated generic material about blogs, study materials, categories, comments and analytics. Those items are not presented here as WeatherWise features.
- One backend screenshot uses the unrelated visible project name AI-STUDY-BUDDY and is omitted.
- Screenshots containing sample passwords or JWT tokens are omitted from this final PDF.
- Keep real API keys, JWT secrets, passwords and MongoDB credentials out of public PDFs and repositories.

Submission checklist

- Fill in student/team details on the cover.
- Add the final demo-video URL in Section 16.
- Add any portal-specific screenshots required by Skill Wallet.
- Check every screenshot for secrets before upload.